GEORG-AUGUST-UNIVERSITÄT GÖTTINGEN IN PUBLICA COMMODA SEIT 1737

Project Thesis

submitted in partial fulfillment of the
requirements for the course "Applied Computer Science"

A comparision of various deep learning based detection systems for multi-object tracking

Lennart Hahner

MatrNr: 16853416

Supervisor: Lukas Steinegger

Georg-August-Universität Göttingen
Institute of Computer Science

ISSN: 1612-6793

March 11, 2026

Georg-August-Universität Göttingen
Institute of Computer Science

Goldschmidtstraße 7
37077 Göttingen
Germany

☎ +49 (551) 39-172000
FAX +49 (551) 39-14403
✉ office@informatik.uni-goettingen.de
www.informatik.uni-goettingen.de

Abstract

[Put your abstract here.]

General structure of an abstract, write 1 to 2 sentences per section

1. A general statement introducing the broad research area of the particular topic being investigated.
2. An explanation of the specific problem (difficulty, obstacle, challenge) to be solved.
3. A review of existing or standard solutions to this problem and their limitations.
4. An outline of the proposed new solution.
5. A summary of how the solution was evaluated and what the outcomes of the evaluation were.

You may find the following resources useful:

- `https://www.grammarly.com/blog/write-an-abstract/`
- `https://www.editage.com/insights/manuscript-structure-how-to-convey-your-most-im`
- More useful links: `https://hps.vi4io.org/teaching/ressources/start`

Declaration on the use of ChatGPT and comparable tools in the context of examinations

In this work I have used ChatGPT or another AI as follows:

- ☑ Not at all
- ☐ During brainstorming
- ☐ When creating the outline
- ☐ To write individual passages, altogether to the extent of 0% of the entire text
- ☐ For the development of software source texts
- ☐ For optimizing or restructuring software source texts
- ☐ For proofreading or optimizing
- ☐ Further, namely: -

I hereby declare that I have stated all uses completely.
Missing or incorrect information will be considered as an attempt to cheat.

Contents

List of Tables iv

List of Figures v

List of Listings vi

List of Abbreviations vii

1 Introduction 1

2 Related work 1
2.1 Overview of modern detection systems . . . 1
2.2 Integration of detection systems in tracking systems . . . 2

3 Experimental setting 4
3.1 Datasets . . . 4
3.1.1 MOT15 Dataset collection . . . 4
3.1.2 MOT20 Dataset collection . . . 5
3.2 Detection systems . . . 6
3.2.1 Faster R-CNN . . . 6
3.2.2 Mask R-CNN . . . 6
3.2.3 YOLOv1-12 . . . 6
3.2.4 DETR . . . 7
3.3 Multi-object-tracking systems . . . 7
3.3.1 SORT . . . 7
3.3.2 DeepSORT . . . 8
3.4 Evaluation metrics . . . 8
3.4.1 Metrics . . . 8
3.4.2 Key Performance Indicators . . . 8

4 Benchmark results 9
4.1 Presentation of various benchmark results . . . 9
4.2 Analysis . . . 9

5 Discussion 10

A Additional Materials A1

List of Tables

List of Figures

1 Object detection systems over time 2
2 4
3 5

List of Listings

List of Abbreviations

HPC High-Performance Computing

1 Introduction

2 Related work

2.1 Overview of modern detection systems

Edozie et al. **edozie** provide a comprehensive review of the development of object detection techniques over the past 26 years. In addition to describing the evolution of deep learning–based object detection systems, they compare these approaches with classical object detection methods that do not rely on deep learning.

Their survey presents a broad overview of relevant datasets, methods, and research contributions over this period. In addition to classical detection methods, they also review modern deep learning–based object detectors, providing a clear overview of the current state of the art.

In their work, they propose several ways to categorize object detection systems. One common categorization distinguishes between one-stage and two-stage detectors. Two-stage detectors first generate region proposals and subsequently classify these proposals into object categories. In contrast, one-stage detectors do not explicitly generate region proposals but instead directly predict object classes and bounding boxes in a single step. Due to this simplified processing pipeline, one-stage detectors generally achieve higher inference speeds than two-stage detectors.

However, this categorization does not explicitly account for newer architectures such as transformer-based models, e.g., DETR <**empty citation**> or multi-path convolutional neural networks such as PANet <**empty citation**> Therefore, the authors additionally classify detection architectures into four categories: traditional detectors, which include non–deep-learning-based methods; conventional deep learning detectors, primarily consisting of two-stage detection systems; hybrid or bridging methods, which include multi-path detection systems; and advanced deep learning methods, which include one-stage detectors and transformer-based models. Figure 1 illustrates an overview of different object detectors, their respective categories, and their year of development, as presented in **edozie**.

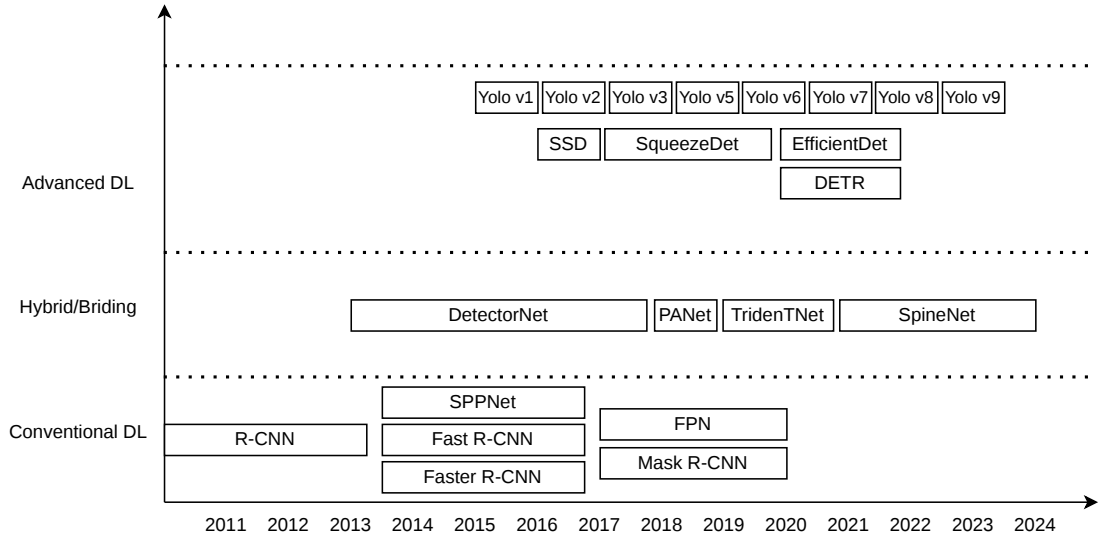

Figure 1: This graphic summarizes the development of object detection systems over recent years, including R-CNN <**empty citation**> SPP-Net <**empty citation**> Fast R-CNN <**empty citation**> Faster R-CNN <**empty citation**> FPN <**empty citation**> Mask R-CNN <**empty citation**> DetectoRS <**empty citation**> PANet <**empty citation**> TridentNet <**empty citation**> SpineNet <**empty citation**> SSD <**empty citation**> SqueezeDet <**empty citation**> EfficientDet <**empty citation**> and DETR <**empty citation**>

Other frameworks, such as open-vocabulary detection and 3D object detection, are not included in the figure but are discussed and categorized in **edozie**.

Similarly, Zou et al. **zou** present a comprehensive survey of object detection methods. They also distinguish between one-stage and two-stage detectors and provide additional insights into modern transformer-based detection architectures. Furthermore, they compare different detection methods, as illustrated in Figure 1, based on their historical development, architectural design, and performance. Performance is evaluated using the mean Average Precision (mAP) metric on benchmark datasets such as VOC07 and MS COCO. Their results show that modern detection systems achieve mAP values exceeding 70%.

Neha et al. **neha** provide an additional overview of object detection systems based on both classical and deep learning approaches. They summarize relevant datasets, evaluation metrics, and detection algorithms, and discuss the strengths and limitations of various deep learning–based detectors. Furthermore, they present benchmark results for representative detection models such as R-CNN and YOLO.

2.2 Integration of detection systems in tracking systems

Li et al. **li** provide a comprehensive overview of modern multi-object tracking (MOT) systems and their integration with object detection methods. They categorize MOT approaches into three main groups: detection-based methods, single-object tracking–based methods, and segmentation-based methods. Detection-based methods first detect objects using an object detector and subsequently associate these detections across frames us-

ing motion and appearance features. In contrast, single-object tracking–based methods use individual object trackers to follow multiple objects across frames. Segmentation-based methods, on the other hand, track objects using pixel-level segmentation instead of bounding boxes, enabling more precise object localization.

The authors emphasize that the performance of multi-object tracking systems strongly depends on the accuracy of the underlying object detector. Modern tracking systems commonly use detectors such as YOLO, CenterNet, and DETR. Their survey also compares multiple tracking methods and shows that modern approaches, such as ByteTrack and BoT-SORT, achieve MOTA values exceeding 80% on the MOT17 dataset. Both ByteTrack and BoT-SORT rely on YOLO-based object detection models.

Adžemović **adzemovic** presents a similar categorization of tracking methods and introduces an additional paradigm referred to as end-to-end tracking, in which detection and tracking are learned jointly within a single neural network. This category includes transformer-based tracking methods such as TrackFormer and MOTR, which jointly learn object detection and association. The survey highlights the performance advantages of end-to-end tracking systems, particularly MOTRv2 <**empty citation**> which achieves state-of-the-art tracking performance. In contrast, ByteTrack achieves the best MOTA performance among tracking-by-detection methods.

Bashar et al. **bashar** also identify tracking-by-detection as the dominant paradigm in modern multi-object tracking systems, where objects are first detected and then associated across frames. Recent approaches increasingly employ transformer-based architectures, such as TrackFormer and TransTrack, which improve tracking performance by modeling temporal relationships between frames. Additionally, graph neural networks, motion models, and Siamese networks are used to enhance object association and identity preservation. The survey reports that modern deep learning–based tracking methods achieve MOTA values of up to 78.3% on benchmark datasets such as MOT17 and MOT20, demonstrating significant improvements in tracking performance.

3 Experimental setting

3.1 Datasets

The choice of dataset is important to correctly reflect the actual performance of different permutations of detection and tracking systems. Especially newer datasets that include night-time scenes, non-urban environments, or highly occluded images can provide useful insights into performance improvements over time. Furthermore, this section highlights the evolution of various dataset collections and challenges.

3.1.1 MOT15 Dataset collection

Different permutations of tracking and detection systems were evaluated on the MOT15 dataset **MOTChallenge2015**. The MOT15 challenge is a dataset framework that contains 5503 training frames and 5783 testing frames, divided into a total of 22 sequences. The dataset includes both 3D and 2D sequences; however, for the current benchmark, only the 2D sequences are considered.

Furthermore, the dataset provides a diverse set of scenes and categorizes the data according to camera movement (static or moving), viewpoint height, and weather conditions. The ground truth annotations are primarily generated through manual human labeling, supported by annotation tools such as the VATIC annotation tool. Some sequences originate from existing datasets, such as the ETH <**empty citation**> dataset and the KITTI <**empty citation**> dataset, which provide their own annotated data. In Figure 2, exemplary ground truth annotations are presented.

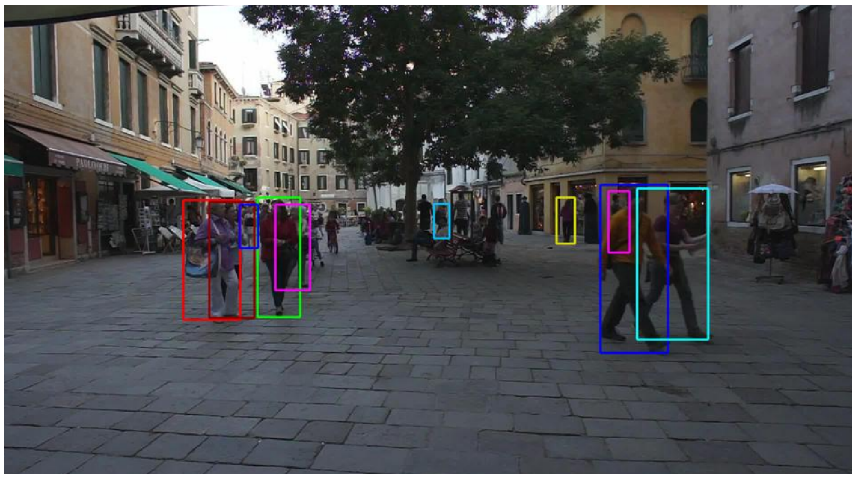

Figure 2: Example scene from the Venice-2 sequence, which is included in the MOT15 dataset. The frames show the point of view (POV) of a static camera in the inner city of Venice. The bounding boxes represent the ground truth annotations provided by the MOT15 challenge. **MOTChallenge2015**

This figure shows one of the targets used in the benchmark. Most sequences, such as KITTI-17, TUD-Crossing, and ETH-Crossing, depict urban environments. The dataset mainly consists of scenes in which multiple pedestrians are visible. As a result, occlusion is common due to overlapping pedestrians. Many sequences originate from datasets such as ETH-Bahnhof, ETH-Sunnyday, and ETH-Pedcross2.

As previously mentioned, the dataset is divided into two parts: the training set and the testing set. While the training and testing sets contain data from similar environments, they are recorded at different times. For example, the KITTI-17 sequence is from 2017 and is located in the training set, while the KITTI-19 sequence is from 2019 and is located in the testing set. This structure makes it possible to train and evaluate detection and tracking models under similar but temporally distinct conditions. Since this report focuses on pre-trained models, the evaluation is independent of whether the training or testing dataset is used.

3.1.2 MOT20 Dataset collection

Since the MOT15 dataset primarily includes daytime scenes with moderate crowd density, an additional dataset from a newer MOTChallenge benchmark is considered. The MOT20 dataset **dendorfer2020mot20benchmarkmultiobject** includes scenes recorded during both daytime and nighttime, and in significantly more crowded environments. This provides additional challenges for detection and tracking systems and allows evaluation under more difficult and realistic real-world conditions.

The MOT20 dataset consists of a total of 8 sequences, split into 4 training and 4 testing sequences. The training set contains 8,931 frames, while the test set contains 4,479 frames. Compared to the MOT15 challenge, the pedestrian density per image has increased substantially, reaching peak densities of up to 246 pedestrians per frame. Furthermore, the dataset contains approximately three times more bounding box annotations than the MOT17 dataset, significantly increasing the complexity of the tracking task. This increase in annotation density and scene complexity is visible in the ground truth example shown in Figure 3.

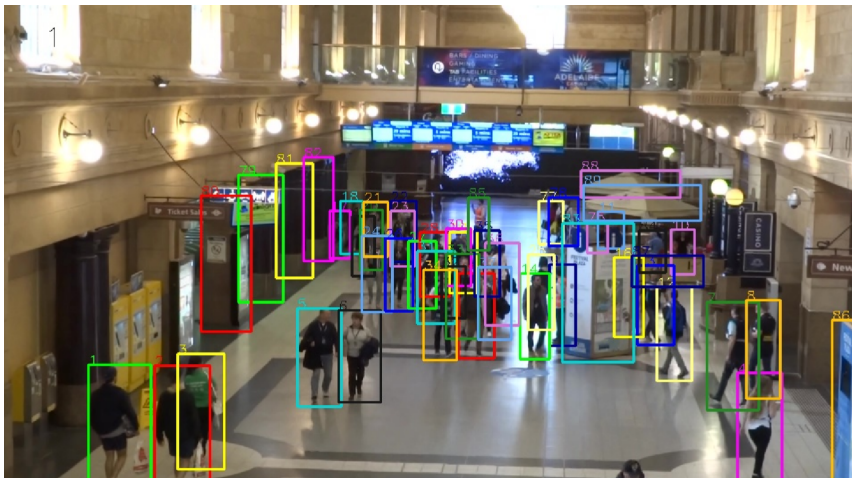

Figure 3: Example scene from the MOT20 dataset. The frames show the point of view of a static camera in a crowded indoor public environment. The bounding boxes represent the ground truth pedestrian annotations provided by the MOT20 challenge. The high number of overlapping annotations illustrates the increased scene complexity and crowd density. **dendorfer2020mot20benchmarkmultiobject**

The high pedestrian density introduces additional challenges for MOT systems, as frequent occlusions between individuals occur. This makes it more difficult to maintain correct identity assignment and to associate object tracks consistently across multiple frames. As illustrated in Figure 3, the dataset includes both indoor and outdoor scenes,

recorded from elevated static camera viewpoints. These scenes contain a wide range of environmental conditions, including daytime and nighttime recordings, varying illumination conditions, and different crowd dynamics.

Additionally, all sequences are recorded in high resolution, typically up to 1920×1080 pixels, allowing precise spatial localization of pedestrians. The dataset provides manually annotated ground truth bounding boxes, including unique object identities and visibility information. This enables a detailed evaluation of tracking performance, particularly in challenging scenarios with partial and full occlusions.

3.2 Detection systems

The benchmark includes different detection systems and runs each on the presented dataset frameworks MOT17 and MOT20. This section should provide an overview of the detailed differences of the detections system.

3.2.1 Faster R-CNN

Fast **R-CNN!** (**R-CNN!**) (2014) **fast-r-cnn** and Faster R-CNN (2016) **faster-r-cnn** build upon one of the earliest deep learning–based object detection systems, R-CNN **r-cnn** (2013). The original R-CNN approach first generates region proposals from an input image using the selective search algorithm **felzenszwalb**. Each proposed region is then passed through a CNN to extract features and classify the object within the region. Since a separate forward pass through the CNN is required for every region proposal, the method does not share computations between proposals and is therefore computationally expensive **fast-r-cnn**. Fast R-CNN addresses this limitation by processing the entire image with a CNN only once. Region proposals generated by selective search are then mapped onto the resulting feature map, where **RoI!** (**RoI!**) pooling extracts fixed-size feature representations for classification and bounding box regression. Faster R-CNN further improves this architecture by replacing the selective search algorithm with a **RPN!** (**RPN!**). The RPN generates candidate object regions directly from the feature maps and predicts both class probabilities and bounding box coordinates, which are subsequently refined by the Fast R-CNN detection network **faster-r-cnn**. All of these models are considered as convolutional deep-learning models visible in figure 1

3.2.2 Mask R-CNN

Another improvement of the R-CNN is the Mask R-CNN. Beside the default output of Faster R-CNN we a class label and a bounding box offset, Mask R-CNN adds an output object mask. Still it adopts the same two-phase procedure as Faster R-CNN but enables with the prediction of masks simultanously segmntation and detection, resulting in more accurate predictions. It is the latest advancement of the R-CNN model. Mask R-CNN is implemennted in the benchmark application using the detectron2 framework <**empty citation**>

3.2.3 YOLOv1-12

The YOLO detection system comes in various version, where each represents one step of improvements. Compared to Faster R-CNN, YOLO does not generate region proposal first, instead it tackles object tracking as a single regression problem, straight from image

pixels to bounding box coordinates and class probabilities. Initally YOLO divides the input image in an $N \times N$ grid. Each cell predicts bounding boxes, confidence scores and class probabilities. Making it a realtime single-stage detection system. **yolo** Further advancements of YOLO progressively addressed its limitations like for example the detection accuracy espacilly for small ones. One advancment was introduced by YOLOv2 a new 30-layer architecture with anchor boxes from Faster R-CNN and batch normalization to speed up convergence and enhance model performance. **yolov2** YOLOv3 then used different scales for detections for espacially small object to captures essential features at different resolutions. **yolov3** Later models such as YOLOv4 **yolov4** and YOLOv5 **yolov5** focused on architectural optimizations, improved feature aggregation, and more efficient training pipelines. From YOLOv6 **yolov6** onward, the focus shifted toward scalability and deployment efficiency, with techniques such as improved label assignment, self-distillation, and enhanced backbone structures. More recent versions, including YOLOv9 **yolov9** and YOLOv10 **yolov10**, introduced innovations such as programmable gradient information and the removal of non-maximum suppression to reduce latency. The latest models, YOLOv11 **yolov11** and YOLOv12 **yolov12**, further advance the architecture through attention mechanisms and optimized feature aggregation networks, achieving state-of-the-art detection accuracy while maintaining real-time inference speeds

The presented benchmark implements Yolov3, -v5, -v8, -v9, -v10, -v11, -v12 and -v26. While Yolov26 is the latest implementation of YOLO from ultralytics. **bibid** That introduces an edge-optimized model which introduces stable and faster inference using the MuSGD (SGD and Muon hybrid) optimizer **yolov26**.

3.2.4 DETR

Besides having a two-stage detection system with Faster R-CNN, we got a one-stage detection system with YOLO now we take a closer look at transformer based detection systems as of DETR **detr**. The DETR networks uses a CNN as a backbone feature extractor, than passes the extracted features to transformer encoder-decoder model. Each encoder layer has a standard architecture and consists of a multi-head self-attention module and a feed forward network (FFN). Essentially the encoder produces object queries that represent positional encoding from the input features. These are then independtly decoded into box coordinates and class lables by the feed forward network (FFN). The FFN predicts the normalized center coordinates, height and width of the box w.r.t. the input image, and the lin- ear layer predicts the class label using a softmax function. DETRs architecture benefits from it's simplitfy and can be extended to other tasks as of object segmentation.

3.3 Multi-object-tracking systems

The benchmark measures the performance of various permutations of detection and tracking systems, the following presents SORT and DeepSORT which are both implemented in the given benchmark.

3.3.1 SORT

SORT! (**SORT!**) is a tracking system introduced in 2017 that, as the name suggests, focuses on online and real-time tracking. In an online setting, the tracker processes frames sequentially and only has access to the current and previous frames while tracking detected objects in real time. The objects are typically detected by an underlying detection

system, such as one of the detectors presented previously. SORT predicts the next possible position of each object in the subsequent frame and associates the new detections with existing tracks based on their bounding boxes. The prediction of the next object position is performed using a Kalman filter framework. Subsequently, SORT computes the **IOU!** (**IOU!**) distance between each detection from the chosen detector and the predicted bounding boxes generated by the Kalman filter. This matching step is referred to as the assignment problem. The assignment between detections and predicted tracks is solved optimally using the Hungarian algorithm. A track is terminated if the corresponding object is not detected for a predefined number of consecutive frames. SORT is implemented for the benchmark and combined with all the named detection systems.

3.3.2 DeepSORT

DeepSORT! (**DeepSORT!**) builds upon the SORT framework for multi-object tracking. While SORT relies solely on motion information for track association, DeepSORT incorporates appearance-based matching using deep learning. Specifically, DeepSORT introduces a CNN-based feature extractor that generates an appearance embedding for each detected object. These appearance features are used in addition to the motion model to improve the reliability of the data association step. In SORT, tracks may be incorrectly swapped when objects become occluded or when multiple objects with similar motion patterns are present. By incorporating visual appearance information, DeepSORT can better distinguish between objects with similar trajectories and therefore significantly reduces identity switches during tracking.

3.4 Evaluation metrics

To measure and compare the performance of the different permutations of detection and tracking systems we compute the given metrics used for the MOT-Challange <**empty citation**>

3.4.1 Metrics

IDF1

$$IDF1 = \frac{|IDTP|}{|IDTP| + 0.5|IDFN| + 0.5|IDFP|} \tag{1}$$

IDF

$$IDF = \frac{|IDTP|}{|IDTP| + 0.5|IDFN| + 0.5|IDFP|} \tag{2}$$

IDR

Recall

Precision

3.4.2 Key Performance Indicators

4 Benchmark results

4.1 Presentation of various benchmark results

4.2 Analysis

5 Discussion

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

A Additional Materials